PROMETHEUS + GRAFANA

OBSERVABILITY HANDS-ON COMMAND EXECUTION GUIDE

macOS + Windows | 2.5-hour classroom practical

Start here
This practical continues from Docker. Docker Desktop should already be installed and running. We build a small near-real DevOps monitoring setup: CI/CD pipeline + application metrics → Prometheus → Grafana.

1. What we are building

```
Pipeline / App :8000/metrics ■■scrape■■> Prometheus :9090 ■■PromQL■■> Grafana :3000
                        cAdvisor :8080 ■■container metrics■■> Prometheus
```

Component	What students see
Prometheus	Scraping, targets, time-series metrics, PromQL, GUI
cAdvisor	Exporter-style container metrics
Grafana	Data source, Explore, panels, dashboard
Pipeline App	Build/deployment + application metrics

2. Fundamentals covered

Syllabus	Practical coverage
Introduction + features	Prometheus/Grafana workflow
Architecture	Scrape/evaluation, TSDB, PromQL, exporters, Grafana data source
Installation & setup	Docker Compose
Advantages / limitations	Short instructor discussion
Monitoring using Prometheus	Targets + queries
Dashboard visualization	Grafana Explore + panels
Pipeline monitoring dashboard	CI/CD dashboard
GUI / plugins	Prometheus UI + Grafana data-source/plugin concept

3. 2.5-hour practical flow

Time	Activity	Output
0–10	Docker + files	Lab ready
10–25	Start stack	4 containers
25–45	Prometheus GUI	Targets + metrics
45–65	PromQL	App + pipeline queries
65–85	Grafana GUI	Data source + Explore
85–120	Dashboard	6 panels
120–135	Failure test	DOWN → UP
135–145	Challenge	2 panels
145–150	Cleanup	Lab stopped

Classroom rule: After Docker is installed, commands are almost identical on macOS and Windows. Folder navigation is the main difference.

4. Installation / setup

macOS

```
docker --version
docker compose version
docker run --rm hello-world
```

Windows PowerShell

```
docker --version
docker compose version
docker run --rm hello-world
```

5. Prepare the lab

macOS

```
cd ~/Downloads/prometheus-grafana-observability-lab
pwd
ls
```

Windows PowerShell

```
cd "$HOME\Downloads\prometheus-grafana-observability-lab"
Get-Location
Get-ChildItem
```

```
prometheus-grafana-observability-lab/
■■■ docker-compose.yml
■■■ prometheus/
■   ■■■ prometheus.yml
■■■ pipeline-app/
    ■■■ app.py
    ■■■ Dockerfile
    ■■■ requirements.txt
```

6. Start the stack

```
docker compose up -d --build
docker compose ps
```

Expected: obs-prometheus, obs-cadvisor, obs-pipeline-app, obs-grafana are running.

7. Prometheus architecture + configuration

```
Targets → scrape /metrics → Prometheus TSDB → PromQL → Grafana
```

Open **prometheus/prometheus.yml**. The three jobs are **prometheus**, **cadvisor** and **pipeline-app**. Inside Docker Compose, service names are used as hostnames.

```
scrape_interval: 5s

job_name: prometheus      target: prometheus:9090
job_name: cadvisor        target: cadvisor:8080
job_name: pipeline-app    target: pipeline-app:8000
```

8. Application metrics

Open **http://localhost:8000/health** → OK. Then **http://localhost:8000/metrics** and find the application/pipeline metrics.

```
app_requests_total
app_request_duration_seconds
app_service_health
pipeline_runs_total
pipeline_duration_seconds
pipeline_last_run_success
pipeline_active_builds
deployment_status
```

9. Prometheus GUI - Targets

Open **http://localhost:9090** → **Status** → **Targets** → confirm prometheus, cadvisor and pipeline-app are **UP**. Open pipeline-app target details and note the /metrics endpoint.

Checkpoint: all three targets UP.

10. Prometheus GUI - PromQL

```
up
up{job="pipeline-app"}
app_requests_total
pipeline_runs_total
pipeline_runs_total{status="success"}
rate(app_requests_total[1m])
rate(pipeline_runs_total[5m])
sum by (status) (rate(pipeline_runs_total[5m]))
```

Teach while executing: metric → label filter → rate() → aggregation. Switch between table/graph views.

11. Prometheus GUI - real pipeline KPIs

```
100 * sum(rate(pipeline_runs_total{status="success"}[5m])) / sum(rate(pipeline_runs_total[5m]))

rate(pipeline_duration_seconds_sum[5m]) / rate(pipeline_duration_seconds_count[5m])

pipeline_last_run_success
pipeline_active_builds
```

12. cAdvisor - container monitoring

Open **http://localhost:8080** and **/metrics**. Back in Prometheus, run:

```
container_cpu_usage_seconds_total
container_memory_working_set_bytes
```

Instructor point: application metrics come from instrumentation; container resource metrics come from an exporter-style component (cAdvisor); Prometheus scrapes both.

13. Grafana GUI - login

Open **http://localhost:3000**. Login **admin / admin**. If prompted, set a lab password.

14. Grafana GUI - Prometheus data source

1. **Connections** → **Add new connection / Data sources** → search **Prometheus**.
2. Select Prometheus.
3. URL: **http://prometheus:9090**.
4. **Save & test**.

Important: Do not use localhost here. Grafana is inside its own container; Prometheus is reached using the Docker Compose service name.

15. Grafana GUI - Explore

Open **Explore** → select the Prometheus data source → execute:

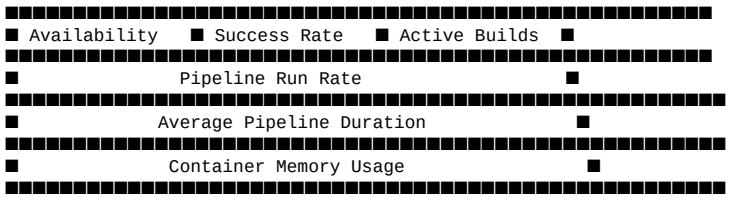
```
up
rate(app_requests_total[1m])
sum by (status) (rate(pipeline_runs_total[5m]))
```

Explore = investigate/query. Dashboard = visualize continuously.

16. Grafana dashboard - create panels

Panel	Query	View
Service Availability	up{job="pipeline-app"}	Stat
Pipeline Success Rate	100 * sum(rate(pipeline_runs_total{status="success"}[5m])) / sum(rate(pipeline_runs_total[5m]))	Stat
Pipeline Run Rate	sum by (status) (rate(pipeline_runs_total[5m]))	Time series
Avg Pipeline Duration	rate(pipeline_duration_seconds_sum[5m]) / rate(pipeline_duration_seconds_count[5m])	Time series
Active Builds	pipeline_active_builds	Stat
Container Memory	sum by (name) (container_memory_working_set_bytes{name!=""})	Time series

17. Dashboard layout + behavior



Set dashboard refresh to about 5s/10s and save as **DevOps Pipeline & Service Monitoring**.

18. Generate realistic activity

Open **http://localhost:8000/simulate** a few times. Return to Grafana and observe request activity, pipeline runs, success/failure, duration and active builds.

The dashboard should answer: **Is the service UP? Are deployments succeeding? How often are builds running? Are builds getting slower? Is container usage increasing?**

19. Failure simulation

```
docker compose stop pipeline-app
# wait 10–15 seconds
# Prometheus → Status → Targets → pipeline-app DOWN

up{job="pipeline-app"}
# expected: 0
```

Return to Grafana and observe the Service Availability panel change.

```
docker compose start pipeline-app
# wait a few seconds → target UP

up{job="pipeline-app"}
# expected: 1
```

This demonstrates: service failure → scrape failure → Prometheus state → Grafana visualization.

20. Final student challenge

```
sum(pipeline_runs_total)
sum(pipeline_runs_total{status="failed"})

deployment_status{environment="production"}
```

Create two additional dashboard panels and choose appropriate visualizations.

21. Quick fixes

Error / symptom	First thing to do
docker not found	Start Docker Desktop; reopen terminal
Docker daemon error	Wait until Docker Desktop is Ready
Port busy	Stop conflict or change host port
Prometheus target DOWN	docker compose ps + docker compose logs <service>
Grafana cannot connect	Use http://prometheus:9090
No PromQL data	Target must be UP; wait for several scrapes
cAdvisor metrics missing	Check cAdvisor logs; continue with app metrics
YAML parse error	Spaces, not tabs; restore prometheus.yml
Windows .yaml.txt	Enable file extensions

22. Cleanup

```
docker compose down

# completely fresh lab:
docker compose down -v
```

23. Final checkpoint

■ Docker verified ■ Stack running ■ Targets checked ■ /metrics inspected ■ PromQL tested ■ cAdvisor explored ■ Grafana data source connected ■ Explore tested ■ 6 panels created ■ DOWN/UP test ■ Challenge panels ■ Cleanup